Arquitetura Cloud-Native Segura e de Baixo Custo no Microsoft Azure: Implantação e Validação do Ambiente lab-popovici-01

Nickolas Corazza Alves

FIAP, Disciplina Cloud Public (GS), rm562265@fiap.com.br
Professor Eduardo Popovic
Av. Paulista, 1106 - 7º andar - Bela Vista, São Paulo - SP, 01311-000
São Paulo/SP, Brasil

Resumo – Este trabalho descreve a implantação manual, com a Azure CLI, de uma arquitetura cloud-native no Microsoft Azure composta por Azure Container Apps, Azure Container Registry (ACR), identidades gerenciadas, rede virtual com DNS privado, PostgreSQL Flexible Server, Key Vault, Storage Account, Log Analytics, Application Insights e Azure Communication Services. As decisões técnicas são relacionadas a um cenário de negócio fictício: o de uma startup de software como serviço (SaaS) que precisa proteger dados pessoais, eliminar credenciais do código e controlar custos. O ambiente foi implantado na região Brazil South, validado por um checklist de 16 itens e teve custo observado de US$ 2,05 em sete dias. São relatados oito desvios em relação ao roteiro, uma falha real de implantação e as limitações do desenho, que o caracterizam como laboratório, e não como ambiente de produção.

I. Introdução

Plataformas de contêineres gerenciadas permitem que pequenas empresas publiquem aplicações sem operar a infraestrutura subjacente. O Azure Container Apps é uma plataforma *serverless* para executar aplicações em contêineres, com escalonamento automático e integração a redes virtuais existentes [1]. Publicar a aplicação, porém, é apenas parte do problema: é preciso definir como ela se autentica em outros serviços, onde ficam os segredos, como o banco de dados é isolado e como a operação é monitorada.

O objetivo deste trabalho é implantar, passo a passo, uma arquitetura que responda a essas questões e registrar, com evidências, o que foi feito, o que divergiu do roteiro [2] e quais são as limitações da solução. A implantação ocorreu em 01/10/2026, em uma assinatura Azure for Students, com a Azure CLI 2.90.0 no Azure Cloud Shell (PowerShell). As 25 capturas de tela que servem de evidência estão no repositório do projeto [3].

II. Cenário de Negócio

Adotou-se um cenário **fictício**, criado para fins acadêmicos. Uma startup brasileira de SaaS, com capital limitado e equipe de infraestrutura reduzida, precisa publicar uma aplicação web que armazena dados pessoais de clientes, envia e-mails transacionais e é operada com o menor esforço possível. Por tratar dados pessoais, está sujeita à Lei Geral de Proteção de Dados (LGPD, Lei nº 13.709/2018) [4]. A Tabela I relaciona cada requisito à decisão técnica adotada. A conformidade com a LGPD depende também de processos e políticas organizacionais; este trabalho trata apenas de controles técnicos de rede e identidade.

Tabela I – Requisitos de Negócio e Decisões Técnicas

Requisito	Decisão técnica
Proteger dados pessoais	PostgreSQL com acesso público desabilitado, em sub-rede delegada, com DNS privado
Evitar vazamento de credenciais	Identidades gerenciadas, Key Vault com RBAC, ACR sem usuário administrador
Menor privilégio	Identidades separadas para a aplicação e para o *job*
Pagar só pelo que usa	Escala a zero no Container App, PostgreSQL pausável, *job* sob demanda
Notificar clientes	Communication Services com domínio gerenciado
Detectar falhas	Log Analytics, Application Insights e Action Group

III. Arquitetura

A solução reside no Resource Group rg-lab-popovici-01, na região Brazil South, e está organizada em camadas (Tabela II e Fig. 1). A VNet 10.20.0.0/16 contém duas sub-redes: snet-containerapps (10.20.0.0/23), integrada ao Container Apps Environment, e snet-postgresql (10.20.2.0/24), delegada ao PostgreSQL. Fora da VNet ficam o ACR (SKU Basic), o Key Vault, o Storage Account (Standard_LRS), o Log Analytics, o Application Insights e o Communication Services. Duas identidades gerenciadas separadas atendem à aplicação e ao *job*. O inventário final totalizou 19 recursos no Resource Group.

Tabela II – Recursos por Camada

Camada	Recursos
Rede	VNet, duas sub-redes, Private DNS Zone e link com a VNet
Execução	Container Apps Environment, Container App e *job* manual
DevOps	ACR (Basic) com a imagem lab-app:v1
Identidade	Duas identidades gerenciadas (aplicação e *job*)
Dados	PostgreSQL Flexible Server 16 (Standard_B1ms) e banco labdb
Segredos e arquivos	Key Vault (RBAC) e Storage Account (Standard_LRS)
Observabilidade	Log Analytics, Application Insights e Action Group
Comunicação	Communication Services, Email Service e domínio gerenciado

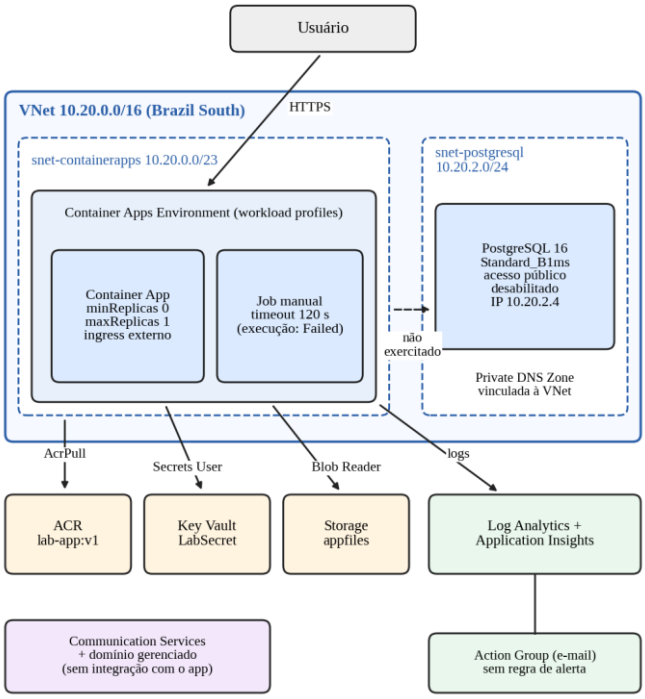


Fig. 1 – Arquitetura implantada. A seta tracejada indica um caminho de rede não exercitado no laboratório

IV. Implementação e Desvios

A implantação seguiu a ordem do roteiro [2]: Resource Group, rede, observabilidade, identidades, ACR, Key Vault, Storage, PostgreSQL, permissões RBAC, ambiente de Container Apps, aplicação e *job*, referência a segredo, comunicação e alertas. Foram usadas as extensões containerapp 1.3.0b5 (preview), communication 1.14.0 e application-insights 1.2.3. A Tabela III resume os oito desvios em relação ao roteiro.

Tabela III – Desvios do Roteiro

#	Desvio	Justificativa
1	Região brazilsouth, em vez de eastus	O SKU Standard_B1ms foi listado só em brazilsouth entre as três regiões testadas; a causa não foi investigada
2	Sufixo no nome do Key Vault	Nomes de Key Vault são globais
3	Funções de dados atribuídas ao usuário	Com RBAC, ser proprietário não dá acesso ao plano de dados (Secrets Officer e Blob Data Contributor)
4	Delegação da snet-containerapps a Microsoft.App/environments e *workload profiles*	Aplicada preventivamente; o comportamento sem ela não foi testado
5	*Job* criado em etapas	A criação direta falhou (Seção IV-A)
6	Segredo referenciado sem versão	A aplicação usa a versão mais recente
7	Parâmetro --name em vez de --database-name e --domain-name	Exigido pela versão da CLI usada
8	E-mail real no Action Group	O endereço fictício do roteiro notificaria terceiros

A. Falha na criação do job

A criação do *job* com --mi-user-assigned e --registry-identity retornou o erro IdentityDoesNotExist, e o recurso não foi criado. A causa **não foi confirmada**; a hipótese é uma limitação da extensão (em preview) ao combinar essas opções em *jobs*, já que o mesmo padrão funcionou para a aplicação. O contorno foi criar o *job* sem registry, atribuir a identidade, configurar o registry com ela e só então atualizar a imagem para a do ACR.

V. Segurança e Identidade

Rede. O PostgreSQL foi implantado com acesso privado (integração com VNet). A CLI confirmou acesso público desabilitado, e o serviço registrou automaticamente o IP 10.20.2.4 na zona DNS privada. Segundo a documentação, nesse modelo as conexões ficam restritas à rede virtual e é necessário usar zonas DNS privadas [5].

Identidade. Com identidades gerenciadas, a aplicação obtém tokens para acessar recursos que usam o Microsoft Entra ID, como Key Vault e Storage, sem armazenar credenciais [6]. As permissões seguem o princípio do menor privilégio (Tabela IV).

Tabela IV – Funções RBAC das Identidades

Identidade	Função	Escopo
Aplicação	Key Vault Secrets User	Key Vault
Aplicação	Storage Blob Data Reader	Storage Account
Aplicação	AcrPull	ACR
Job	AcrPull	ACR

Segredos. O Key Vault usa RBAC, e o segredo LabSecret é referenciado no Container App por identidade (*secretRef*), de modo que o valor permanece no cofre. O ACR foi criado com o usuário administrador desabilitado e o *pull* da imagem ocorre por identidade; a revisão da aplicação ficou *Healthy*, o que confirma esse acesso. No Storage, o acesso anônimo a blobs foi desabilitado e o TLS mínimo é 1.2.

VI. Observabilidade e Custos

O Log Analytics (retenção de 30 dias) centraliza os logs do ambiente, e o Application Insights foi vinculado ao workspace, que também criou automaticamente um Action Group de *Smart Detection*. Um Action Group próprio, com receptor de e-mail, foi criado, mas sem regra de alerta associada ele não dispara notificações. Uma consulta KQL para investigar o *job* falhou por usar um nome de coluna inexistente, e a investigação por logs não foi concluída.

Custos. Foram aplicadas duas medidas reversíveis: a parada do PostgreSQL (a computação deixa de ser cobrada, mas o armazenamento continua, e o Azure reinicia o servidor após 7 dias) e minReplicas igual a 0 no Container App. Entre 02/10 e 08/10/2026, o Azure Cost Management registrou o custo total de US$ 2,05 (R$ 10,70), detalhado na Tabela V, o que equivale a cerca de 0,7% do crédito inicial de US$ 312. Nos seis dias completos o custo diário foi estável, em torno de US$ 0,30. Extrapolado linearmente (cálculo do autor, não previsão do Azure), o ritmo daria cerca de US$ 9 por mês.

Tabela V – Custos Observados (02 a 08/10/2026)

Resource Group	US$	R$	US$/dia*
rg-lab-popovici-01	1,23	6,44	0,18

Resource Group	US$	R$	US$/dia*
Grupo gerenciado (ME_)	0,82	4,25	0,12
Total	**2,05**	**10,70**	**0,30**

Fonte: Azure Cost Management, exportações diária e acumulada de 09/10/2026 (totais coincidentes). *Média dos dias completos (02 a 07/10); o dia 08/10 parece parcial (hipótese). O dia 01/10 não está no período filtrado.

O grupo gerenciado ME_ contém somente o IP público e o *load balancer* e custou US$ 0,12 por dia, mesmo com a aplicação escalada a zero. Isso mostra que escalar a zero não elimina o custo de um ambiente com *ingress* externo integrado à VNet. A exportação agrupa por Resource Group, e não por recurso; portanto, o custo diário do rg-lab-popovici-01 (US$ 0,18) não pode ser atribuído a cada recurso. **Hipótese não confirmada:** a maior parte seria o ACR Basic, que tem tarifa diária fixa, publicada pela Microsoft em US$ 0,167 [7], e não pode ser pausado.

VII. Validação e Resultados

A validação seguiu o checklist de 16 itens do roteiro [2], todos com evidência [3]. A Tabela VI resume os resultados principais. O endpoint HTTPS público respondeu com a página de exemplo do Container Apps.

Tabela VI – Resumo da Validação

Verificação	Resultado
Sub-redes e delegação	Conforme esperado
Link da Private DNS Zone	Completed / Succeeded
PostgreSQL	Ready; acesso público desabilitado
ACR	Tag lab-app:v1 presente
Container Apps Environment	Succeeded, em Brazil South
Container App	Revisão Healthy; responde via HTTPS
Job	Execução registrada; estado final Failed
Communication Services	Succeeded; DKIM, SPF e DMARC verificados
Action Group	Ativo; receptor habilitado

Sobre o job. A execução ficou registrada no histórico, como o roteiro exige, mas terminou como *Failed*. A causa **não foi comprovada**. Uma hipótese plausível é que a imagem de exemplo, um servidor web, não termina sozinha e o *timeout* de 120 s encerrou a execução.

VIII. Limitações e Trabalhos Futuros

O ambiente é um laboratório, e não uma arquitetura de produção. Pelas saídas da CLI: não há redundância de zona, o PostgreSQL está no *tier* Burstable e o Storage é Standard_LRS; a aplicação tem *ingress* externo; Key Vault, ACR e Storage aceitam conexões pela rede pública, com autenticação, e somente o PostgreSQL é privado; o Key Vault tem *soft delete* de 90 dias, mas sem proteção contra purga; o domínio de e-mail foi criado, porém nenhum e-mail foi enviado nem o domínio foi associado ao Communication Services; a imagem de exemplo não se conecta ao banco; e a extensão containerapp e o grupo communication email estavam em preview.

Como trabalhos futuros, propõem-se: *private endpoints* e proteção contra purga; regras de alerta; investigação do *job* por logs; envio de e-mail de teste; exportação dos custos por recurso; automação por *scripts* ou infraestrutura como código; e avaliação do desenho segundo o Azure Well-Architected Framework [8].

IX. Conclusão

O laboratório permitiu implantar, de ponta a ponta, uma arquitetura Azure que combina execução de contêineres, banco de dados em rede privada, autenticação por identidade gerenciada, gestão de segredos, observabilidade e comunicação. No cenário fictício, cada decisão foi ligada a um requisito de negócio: proteção de dados, ausência de credenciais no código, menor privilégio e controle de custos, cujo valor observado foi baixo, embora parte dele seja fixa. O processo também expôs problemas reais, como a disponibilidade de SKU por região, as permissões de plano de dados e a falha na criação do *job*, cuja resolução faz parte do aprendizado. Por ser um laboratório, a solução tem limitações de disponibilidade e exposição que precisariam ser tratadas antes de qualquer uso em produção.

Referências

[1] Microsoft. “Azure Container Apps overview”. Microsoft Learn. Disponível em: https://learn.microsoft.com/en-us/azure/container-apps/overview. Acesso em: 8 out. 2026.

[2] E. Popovici. “Laboratório Azure: Construindo o ambiente lab-popovici-01 – Guia passo a passo”. Material didático, Popovici Tech Academy, 2026.

[3] N. Corazza. “cloudpublic-cp2: evidências do laboratório lab-popovici-01”. GitHub. Disponível em: https://github.com/ncorazza12/cloudpublic-cp2. Acesso em: 8 out. 2026.

[4] Brasil. “Lei nº 13.709, de 14 de agosto de 2018 (Lei Geral de Proteção de Dados Pessoais)”. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm. Acesso em: 8 out. 2026.

[5] Microsoft. “Network with private access (virtual network integration) for Azure Database for PostgreSQL flexible server”. Microsoft Learn. Disponível em: https://learn.microsoft.com/en-us/azure/postgresql/network/concepts-networking-private. Acesso em: 8 out. 2026.

[6] Microsoft. “Managed identities in Azure Container Apps”. Microsoft Learn. Disponível em: https://learn.microsoft.com/en-us/azure/container-apps/managed-identity. Acesso em: 8 out. 2026.

[7] Microsoft. “Azure Container Registry service tiers”. Microsoft Learn. Disponível em: https://learn.microsoft.com/en-us/azure/container-registry/container-registry-skus. Acesso em: 8 out. 2026.

[8] Microsoft. “Azure Well-Architected Framework”. Microsoft Learn. Disponível em: https://learn.microsoft.com/en-us/azure/well-architected. Acesso em: 8 out. 2026.